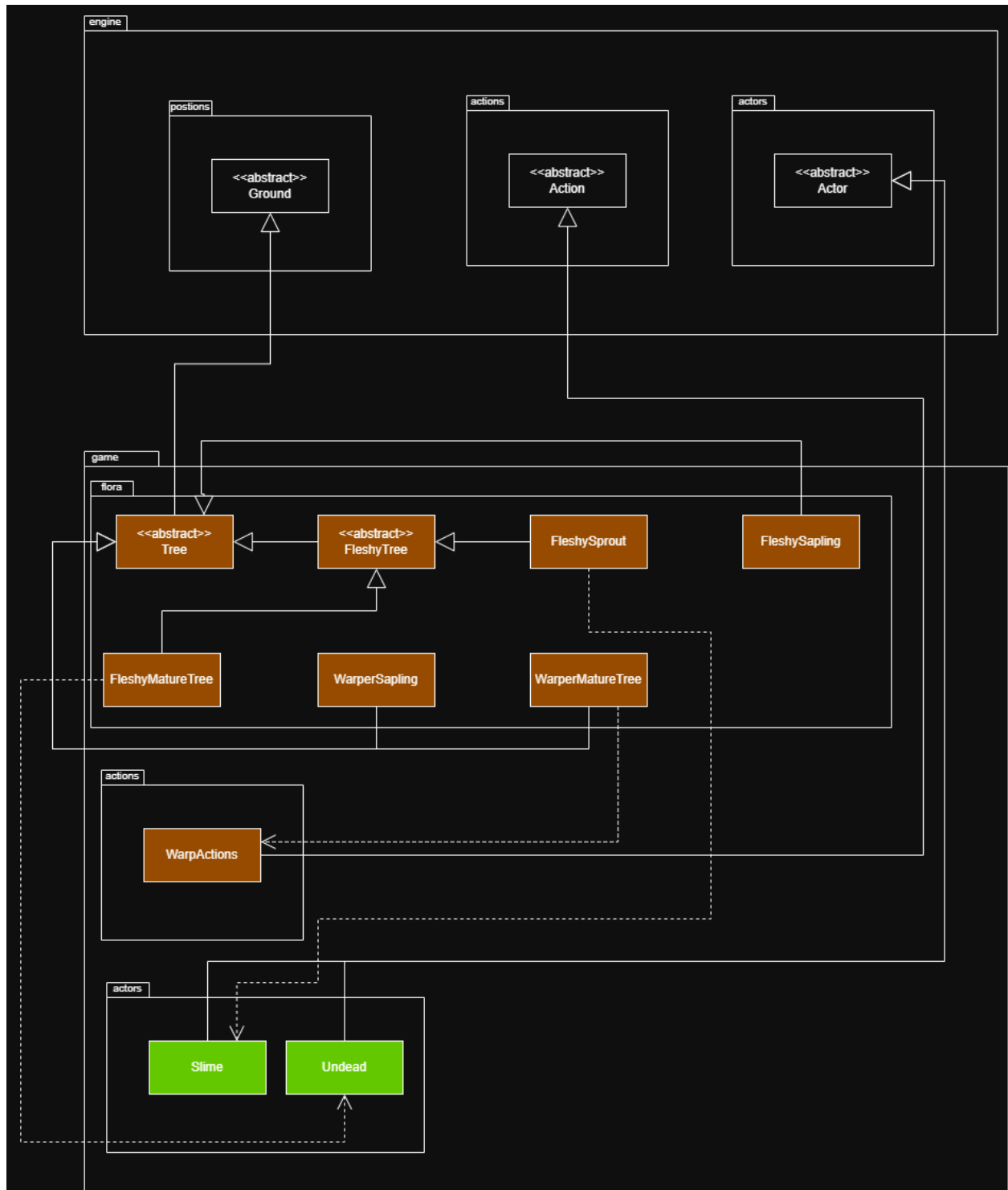


REQ 3 Design Rationale



A. Flora as Ground Subclasses (not Actors)

Design 1: Implement flora (Fleshy Trees and Warper Trees) as subclasses of the abstract Actor class.

Pros	Cons
Actors already have a playTurn() method that can be used to implement flora behaviour each turn.	Flora does not move, attack, or make decisions — these are all Actor responsibilities that flora does not need, violating the Single Responsibility Principle.
	Actors occupy a tile and are tracked as actors on the map, whereas flora should behave as terrain that other actors walk on or near, not as an entity in the actor pool.
	Adds unnecessary overhead: actors require an Inventory, IntrinsicWeapon, and are iterated in the game loop as decision-making entities, none of which flora needs.

Design 2 (Chosen): Implement flora as subclasses of the abstract Ground class, overriding tick(Location) to implement per-turn behaviour.

Pros	Cons
Ground.tick(Location) is already called every turn for each terrain tile, making it the natural point to implement growth and proximity effects without modifying the engine.	Extending an abstract class introduces coupling — changes to the Ground class would propagate to all flora subclasses.
Flora correctly models terrain: it occupies a fixed tile, and actors step near it (on adjacent tiles) rather than interacting with it as another actor.	
Reuses the existing engine architecture	

Chosen: Design 2.

Flora extends Ground because it is terrain, not an agent. This uses the engine as intended and avoids burdening the actor pool with non-actor entities.

B. Enforcing the Single-Action-Per-Turn Rule

The specification states: *"Each flora can only perform ONE action per turn."* For example, a Fleshy Sprout cannot both spawn a Slime and grow into a Sapling in the same turn.

Design 1: Each concrete tree subclass independently manages priority in its own overridden tick() method, using an if/else check.

Pros	Cons
Simple per-class implementation with no shared logic to reason about.	The same <code>if (!proximityEffect) { tryGrow(); }</code> logic is duplicated across every concrete subclass that has both behaviours (DRY violation).
	No structural guarantee that a future developer adding a new stage will remember to enforce the rule — the constraint is implicit, not enforced by design.

Design 2 (Chosen): Implement a Template Method pattern in `Tree.tick()`. The abstract base class defines the algorithm skeleton and subclasses override the hook methods `proximityEffect()` and `tryGrow()`.

`Tree.tick()` skeleton:

```
turnCount++
if (!proximityEffect(location)) :
    tryGrow(location)
```

Pros	Cons
The single-action constraint is encoded once in the abstract class and inherited automatically by all subclasses. No subclass can accidentally violate it.	Subclasses must follow the hook method contract (override <code>proximityEffect</code> and/or <code>tryGrow</code>), which introduces a dependency on the parent's algorithm structure.
Eliminates all code duplication of the priority check (DRY).	
New tree stages can be added by simply extending <code>Tree</code> and overriding the relevant hooks, without touching any existing code (Open/Closed Principle).	

Chosen: Design 2.

The Template Method pattern makes the single-action rule a structural guarantee rather than a convention, enforcing it for all current and future flora stages.

C. Centralising the Proximity-Spawn Logic (FleshyTree)

Both FleshySprout and FleshyMatureTree share identical behaviour: scan adjacent tiles for any actor, and if found, spawn a new creature there. The only difference is which creature is spawned.

Design 1: Implement the adjacency scan and spawn loop independently in both FleshySprout and FleshyMatureTree.

Pros	Cons
Each class is self-contained with no additional abstraction layer.	The adjacency-check-and-spawn loop is duplicated in two classes. If the logic changes (e.g. checking for a specific actor type), both classes must be updated (DRY violation).
	Harder to maintain as the number of fleshy tree stages grows.

Design 2 (Chosen): Introduce an abstract intermediate class FleshyTree that extends Tree. FleshyTree provides a concrete proximityEffect() implementation that delegates the actual spawn to an abstract method spawnActor(Location). Subclasses only implement spawnActor().

Pros	Cons
The adjacency-check logic is written once in FleshyTree (DRY).	Adds one more level to the inheritance hierarchy, slightly increasing depth.
FleshySprout and FleshyMatureTree each have a single responsibility: define what to spawn. The 'how to detect' is separated from the 'what to do' (Single Responsibility Principle).	
Adding a new fleshy tree stage requires only extending FleshyTree and implementing spawnActor() — no changes to existing code (Open/Closed Principle).	

Chosen: Design 2.

FleshyTree is a justified intermediate abstraction: it eliminates real duplication and separates detection logic from spawn logic.

D. FleshySapling Placement in the Hierarchy

FleshySapling is Stage 2 of the Fleshy Tree. Unlike FleshySprout and FleshyMatureTree, it has **no proximity effect** — it only grows.

Design 1: FleshySapling extends FleshyTree, providing an empty or no-op spawnActor() implementation.

Pros	Cons
Groups all three fleshy tree stages under the same parent, reflecting the thematic taxonomy.	FleshyTree declares spawnActor() as abstract, meaning FleshySapling inherits a contract it cannot meaningfully fulfil. Providing an empty implementation violates the Liskov Substitution Principle — a FleshySapling substituted for a FleshyTree would silently do nothing on proximity, breaking the expected spawning contract.

Design 2 (Chosen): FleshySapling extends Tree directly, bypassing FleshyTree.

Pros	Cons
FleshySapling only inherits the tryGrow() hook it actually uses. No spurious abstract method is imposed on it.	The three fleshy stages are not all grouped under a single common parent, which slightly reduces thematic cohesion in the class hierarchy.
Preserves the Liskov Substitution Principle: every FleshyTree instance genuinely has a proximity-spawn effect.	

Chosen: Design 2.

FleshySapling extends Tree directly to uphold LSP. A class should not inherit a contract it cannot honour.

E. WarpAction vs Reusing TeleportAction (REQ 2)

WarperMatureTree must forcibly teleport a nearby worker to a random location. The existing TeleportAction (from REQ 2) also moves actors between locations.

Design 1: Reuse TeleportAction by making WarperMatureTree implement the Teleportable interface, providing getDestinations() to return a random location.

Pros	Cons
Avoids creating a new class for a similar operation.	TeleportAction is designed for player-initiated, device-driven teleportation to a pre-selected destination. Forcing WarperMatureTree (a terrain tile) to implement Teleportable (a device interface) misuses the interface contract.
	getDestinations() in Teleportable is intended to return pre-configured destinations; computing a random location at execution time is semantically different and would produce misleading code.
	The destination for TeleportAction is fixed at construction time; for warping it must be random at execution time — the constructor signatures are fundamentally incompatible.

Design 2 (Chosen): Create a separate WarpAction class that extends Action. It computes a random unoccupied traversable location at execution time and moves the actor there. WarperMatureTree instantiates and executes it directly in proximityEffect().

Pros	Cons
Clear separation of concerns: TeleportAction is player-initiated with a device and side effects; WarpAction is terrain-forced and random. Each class has one responsibility (Single Responsibility Principle).	Introduces a new class rather than reusing an existing one.
WarpAction encapsulates the random-destination logic, keeping WarperMatureTree clean and readable.	
Does not pollute the Teleportable interface with a use case it was not designed for (Interface Segregation Principle).	

Chosen: Design 2.

WarpAction and TeleportAction serve fundamentally different contracts. Reusing TeleportAction would misuse the Teleportable interface and couple flora to the REQ 2 teleportation device system.

Design Summary & SOLID Principles

All flora extends Ground, reusing the engine's tick(Location) lifecycle correctly. The Tree abstract class uses the Template Method pattern to enforce the single-action-per-turn rule structurally, eliminating the need for each subclass to independently manage priority (DRY).

FleshyTree centralises the proximity-check logic shared by FleshySprout and FleshyMatureTree, applying the DRY principle at the intermediate abstraction level. FleshySapling extends Tree directly rather than FleshyTree to preserve the Liskov Substitution Principle.

WarpAction is a separate class from TeleportAction to uphold the Single Responsibility Principle and avoid misusing the Teleportable interface (Interface Segregation Principle). Flora subclasses depend on the abstract Tree hooks rather than on each other, consistent with the Dependency Inversion Principle.